

P1. Echo over TCP in C

Two programs, two terminals, one conversation. You are given code that almost works; making it work is the exercise.

What you should be able to do when you finish

- Write, build and run a TCP client and server in C, and name every system call in the sequence.
- Read an error from the compiler, from `perror` and from the operating system, and turn each one into a fix.
- Explain, with your own measurements, why one send is not one receive.

What you are given

File	What it is	State
P1_echo_server.c	Waits for a client, reads what arrives and sends it back.	Contains three defects
P1_echo_client.c	Sends a message of the size you ask for, waits for it to come back, and prints how long it took.	Correct - use it as reference

What is wrong with it

The starting server does not work. It contains a small number of defects, of three different kinds: one that the compiler rejects, one that appears the first time you run it, and one that appears only when the messages become large. Fix them one at a time and run the program between fixes.

Building and running

```
# the laboratory machines already have everything you need.
# on your own computer, if gcc is missing and you have
# administrator rights:  sudo apt install build-essential

# build both programs
gcc -Wall -o P1_echo_server P1_echo_server.c
gcc -Wall -o P1_echo_client P1_echo_client.c

# terminal 1
./P1_echo_server

# terminal 2
./P1_echo_client 127.0.0.1 9000 1024

# useful while you work
ps -ef | grep P1_echo_server    # is your server still running
echo $?                        # how the last program ended
```

Before you change anything

Run it once, unmodified, and keep the exact text of the first error you see. You will be asked for it, word for word, and you cannot reconstruct it later.

Three stages

Stage one is compulsory and everybody is expected to reach it. Stages two and three are where the marks separate.

STAGE 1 Make it run	
What to do	Fix the three defects. Above each fix, write a comment in the code saying what was wrong, how you noticed it, and why your change is correct.
What to observe	The server prints the port it listens on and how many bytes it receives; the client prints the bytes echoed and the time taken.
What to hand in	The corrected sources, plus a copy of the terminal output of one successful exchange.
STAGE 2 Change one thing and measure	
What to do	Send messages of 10, 1 000, 10 000 and 100 000 bytes. Run each size five times and keep the average.
What to measure	The time reported by the client, and the number of times the server prints that it has received data for each size.
What to hand in	A table with four rows: size, average time in milliseconds, and how many chunks the server needed. One sentence saying whether the time grows in proportion to the size, and why.
STAGE 3 Break it on purpose	
What to do	Three experiments. First, stop the server with Ctrl-C while a client is waiting for its answer. Second, run the client against port 9001, where nothing is listening. Third, run two clients at the same time against the server.
What to observe	The exact message each side prints and how long it takes to appear. The second experiment fails immediately: read what the client says and compare it with the silence of a server that is running but slow.
What to hand in	For each of the three, the message you saw and one sentence explaining the cause. For the third, say which client was served first and what the other one was doing meanwhile.

What you hand in

Piece	Detail
The code	Both sources, compiling without warnings with <code>-Wall</code> , and with your own comments explaining each fix in your own words.
Your numbers	The table from stage two and the messages from stage three, as a plain text file, copied from your terminal and not retyped.
One handwritten page	Scanned or photographed, with your name and the date at the top. One short paragraph for each: what failed and how you knew, what you changed, what you measured.

How it is marked

Level	What it looks like
Not enough	The code does not compile, or the fixes are there with no explanation of why.
Sufficient	The three defects are fixed and explained, and one exchange is shown working.
Good	The above, plus the measurement table with a correct reading of what the numbers show.
Very good	The above, plus the three failure experiments with a correct explanation of each message.

A note on how you work

You may use any tool you like to understand the code. You may not hand in anything you cannot explain. You can be asked in class to justify any line, any comment and any number in your table.